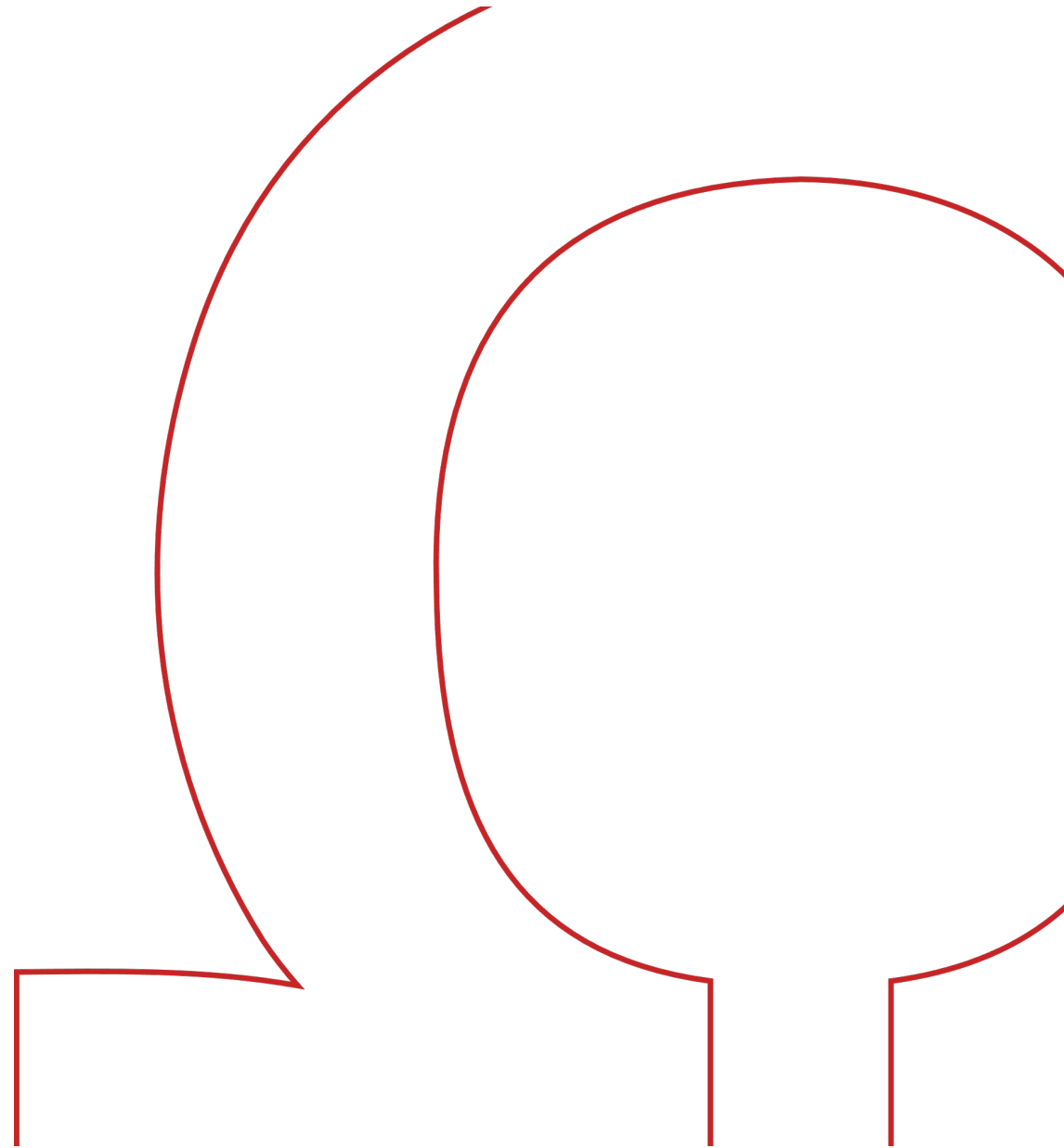


Prog III / A

Software Engineering



Agiles Manifest

Agiles Manifest

Präambel

Agile Softwareentwicklung verfolgt den Ansatz, die Ideen des **Lean Thinking** auf die Erstellung von Software zu übertragen.

Wir erschließen bessere Wege, Software zu entwickeln,
indem wir es selbst tun und anderen dabei helfen.
Durch diese Tätigkeit haben wir diese Werte zu schätzen gelernt:

Individuen und Interaktionen mehr als **Prozesse und Werkzeuge**
Funktionierende Software mehr als **umfassende Dokumentation**
Zusammenarbeit mit dem Kunden mehr als **Vertragsverhandlung**
Reagieren auf Veränderung mehr als das **Befolgen eines Plans**

Das heißt, obwohl wir die Werte auf der rechten Seite wichtig finden,
schätzen wir die Werte auf der linken Seite höher ein.

Kent Beck
Mike Beedle
Arie van Bennekum
Alistair Cockburn
Ward Cunningham
Martin Fowler

James Grenning
Jim Highsmith
Andrew Hunt
Ron Jeffries
Jon Kern
Brian Marick

Robert C. Martin
Steve Mellor
Ken Schwaber
Jeff Sutherland
Dave Thomas

Agiles Manifest

Die 12 Prinzipien des Agilen Manifests

1. Unsere höchste Priorität ist es, den Kunden durch frühe und kontinuierliche Auslieferung wertvoller Software zufrieden zu stellen
2. Anforderungsänderungen selbst spät in der Entwicklung sind willkommen. Agile Prozesse nutzen Veränderungen zum Wettbewerbsvorteil des Kunden
3. Liefere funktionierende Software regelmäßig innerhalb weniger Wochen und Monate und bevorzuge dabei die kürzere Zeitspanne
4. Fachexperten und Entwickler müssen während des Projekts täglich zusammenarbeiten
5. Errichte Projekte rund um motivierte Individuen. Gib ihnen das Umfeld und die Unterstützung, die sie benötigen, und vertraue darauf, dass sie die Aufgaben erledigen
6. Funktionierende Software ist das wichtigste Fortschrittsmaß
7. Die effizienteste und effektivste Methode, Informationen an und innerhalb eines Entwicklungsteams zu übermitteln, ist das Gespräch von Angesicht zu Angesicht
8. Agile Prozesse fördern nachhaltige Entwicklung. Die Auftraggeber, Entwickler und Benutzer sollten ein gleichmäßiges Tempo auf unbegrenzte Zeit halten können
9. Ständiges Augenmerk auf technische Exzellenz und gutes Design fördert Agilität
10. Einfachheit – die Kunst, die Menge nicht getaner Arbeit zu maximieren – ist essenziell
11. Die besten Architekturen, Anforderungen und Entwürfe entstehen durch selbstorganisierte Teams
12. In regelmäßigen Abständen reflektiert das Team, wie es effektiver werden kann, und passt sein Verhalten entsprechend an

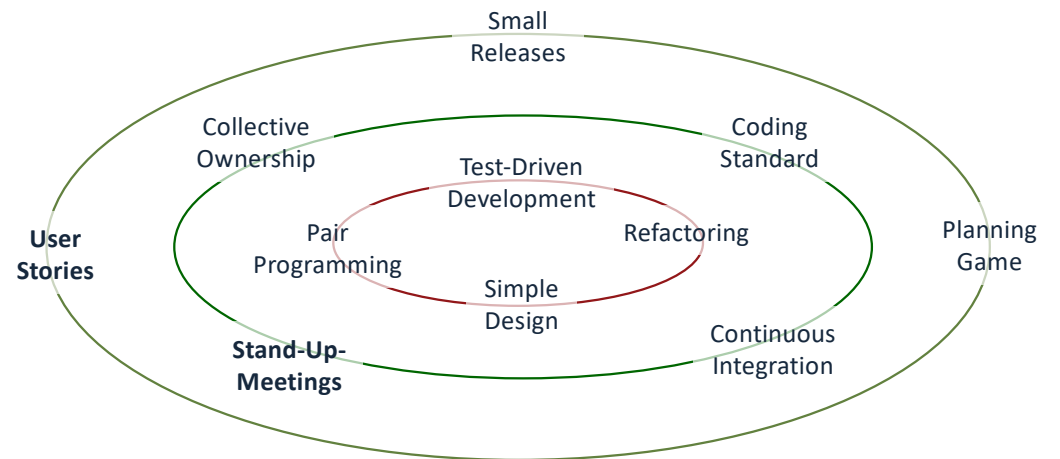
eXtreme Programming (XP)

eXtreme Programming (XP)

Agile Praktiken

- früher Versuch der Übertragung von Lean Thinking auf die Software-Entwicklung
- eine der ersten Sammlungen agiler Praktiken
- Basis für weitergehende Methoden (z.B. SCRUM)
- Grundidee: Identifikation von guten Vorgehensweisen und deren Ausbau bis zum Extrem

Agile Praktiken (Auszug):

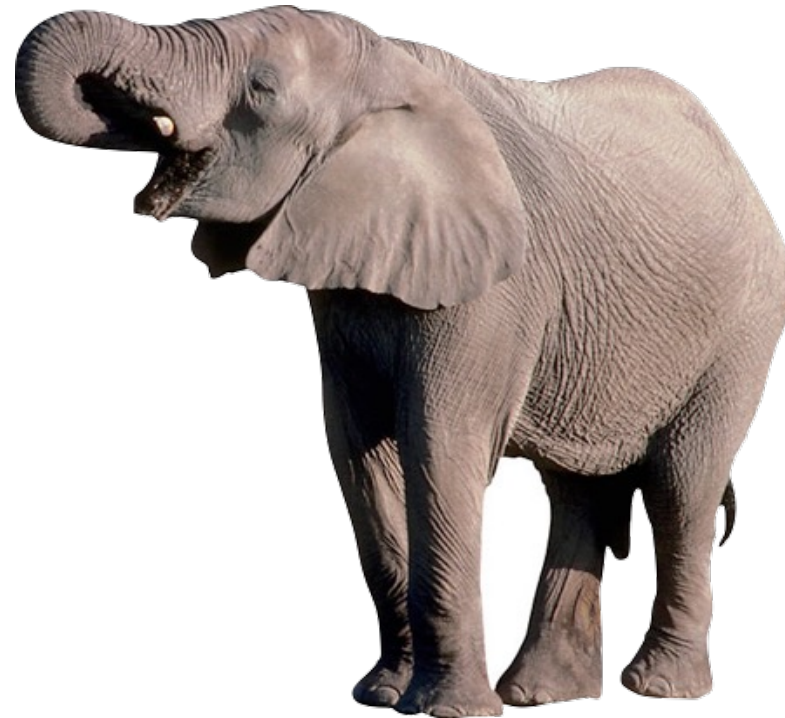


eXtreme Programming (XP)

User Stories

Softwareprojekte sind oft
schwerfällig und
unüberschaubar komplex.

Wie bekommt man
Komplexität in den Griff?



eXtreme Programming (XP)

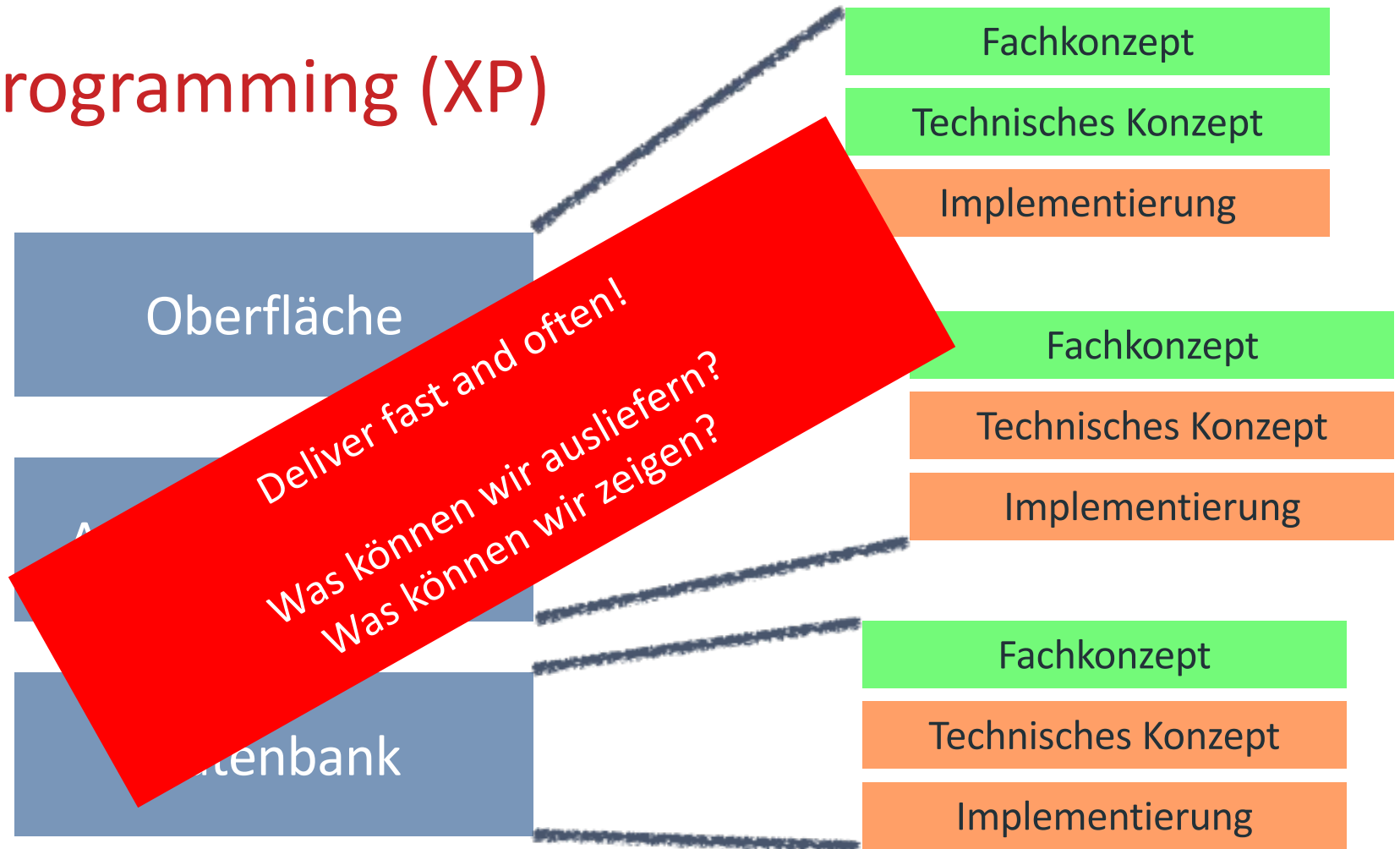
User Stories

Das Zerlegen komplexer Aufgaben in kleine Teilaufgaben hat sich bewährt, daher wird das gesamte Projekt in überschaubare **User Stories** aufgeteilt.



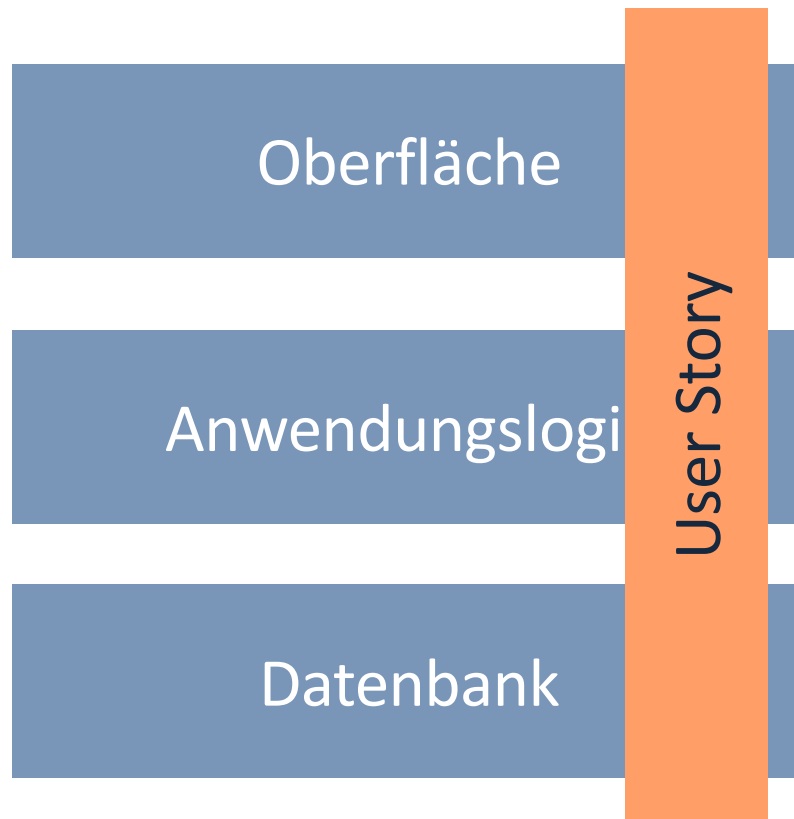
eXtreme Programming (XP)

User Stories



eXtreme Programming (XP)

User Stories



- eine aus Anwendersicht formulierte Anforderung
- Ist keine fertige Lösung, sondern Basis einer Diskussion
- schneidet das System vertikal, nicht horizontal
- Beinhaltet Kriterien, wann ein System die Story erfüllt

eXtreme Programming (XP)

User Stories

Story-Karte

Als **<Rolle>** will ich **<Ziel>**
[, so dass **<Grund für das Ziel>**]

eXtreme Programming (XP)

User Stories

Story-Karte (Rückseite)

Angenommen **<Vorbedingungen>**, wenn
<Aktion>, dann **<Ergebnis>**

eXtreme Programming (XP)

User Stories

I	Independent	User Stories sollten unabhängig voneinander sein.
N	Negotiable	User Stories sollten verhandelbar sein.
V	Valuable	User Stories sollten einen Wert für den Kunden haben.
E	Estimatable	User Stories sollten schätzbar sein.
S	Small	User Stories sollten klein sein.
T	Testable	User Stories sollten testbar sein.

eXtreme Programming (XP)

Stand-Up-Meeting

Meetings dauern oft zu lange, weil

- zwar schon alles gesagt ist
- aber noch nicht von jedem

Warum machen wir sie dann nicht
unbequem, kurz (dafür regelmäßig) und
strukturiert?



eXtreme Programming (XP)

Stand-Up-Meeting

- ✓ Tägliches Treffen
- ✓ Im Stehen abgehalten
- ✓ Bis zu 12 Teilnehmer
- ✓ 15 Minuten Timebox



eXtreme Programming (XP)

Stand-Up-Meeting

Jeder Teilnehmer beantwortet dem gesamten Team drei Fragen:

- Was habe ich seit dem letzten Stand-Up getan?
- Was werde ich bis zum nächsten Stand-Up tun?
- Was behindert mich?

Hilfreich:

Verwendung eines Taskboards (s. Bild)

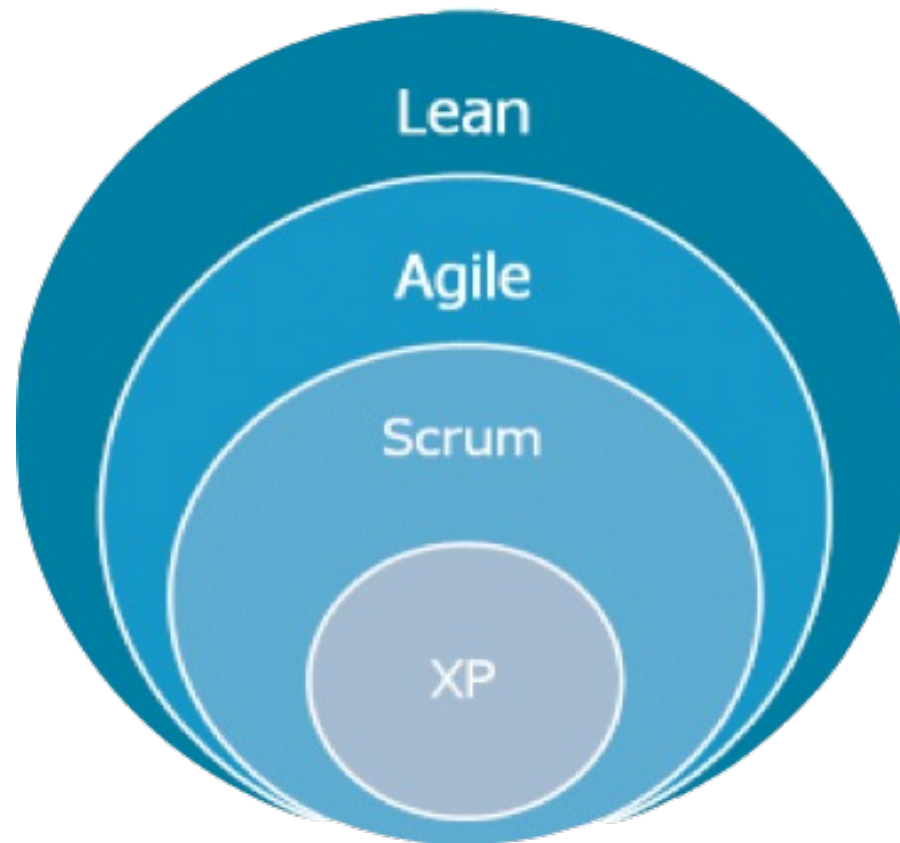
Diskussionen sind nicht erlaubt, nur Rückfragen. Detailklärungen im Anschluss daran bilateral (d.h. nur zwischen den Beteiligten).



Scrum

Scrum

Einordnung



ohm

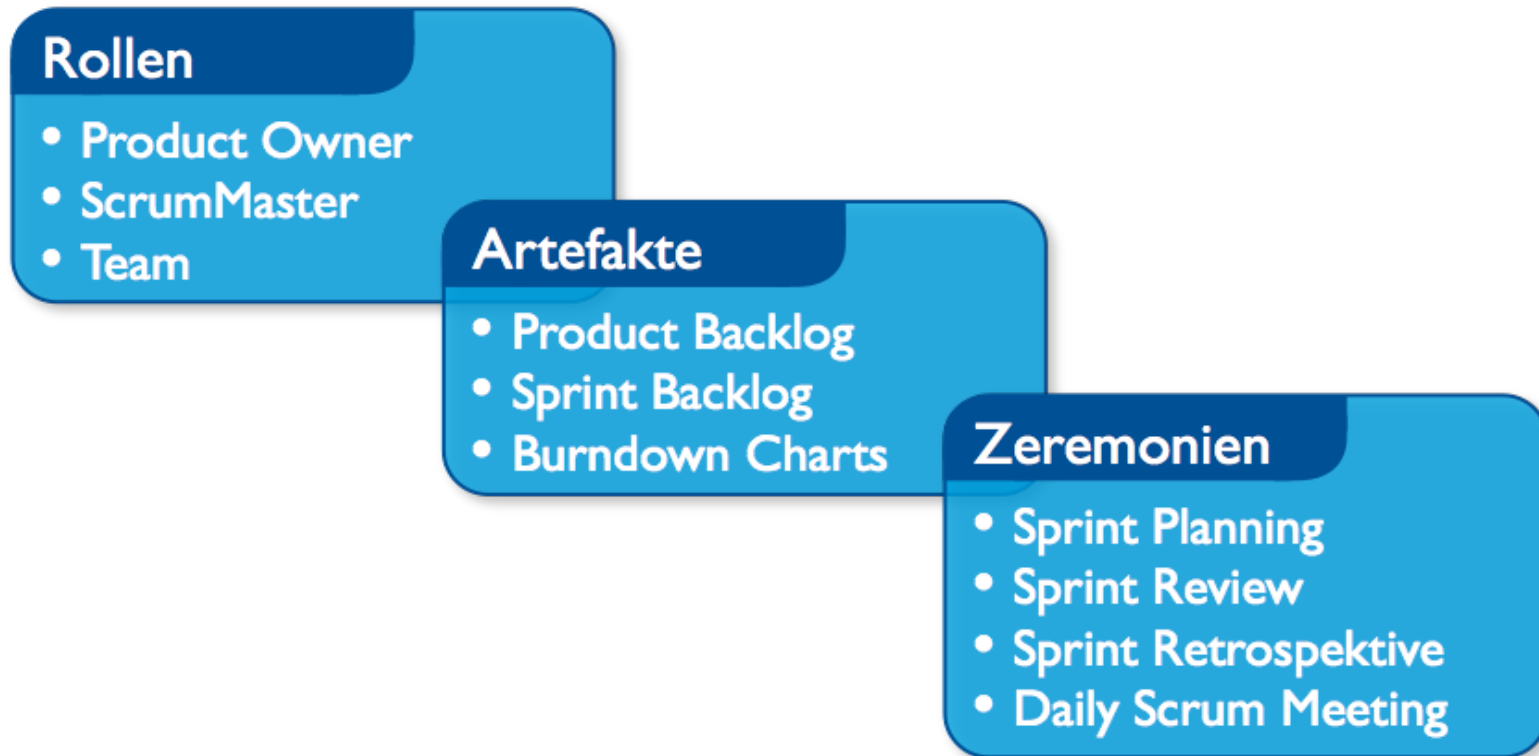
Scrum

Begriff



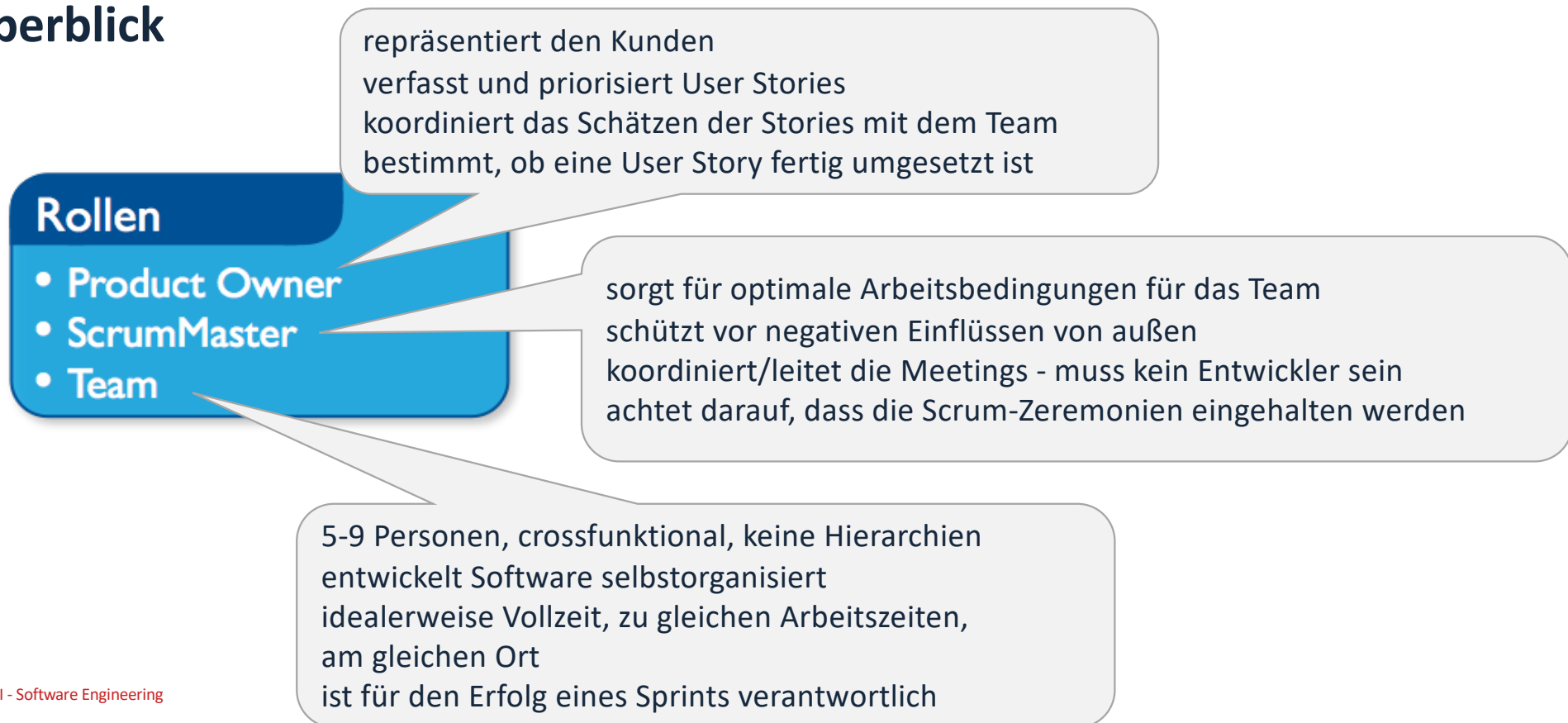
Scrum

Überblick



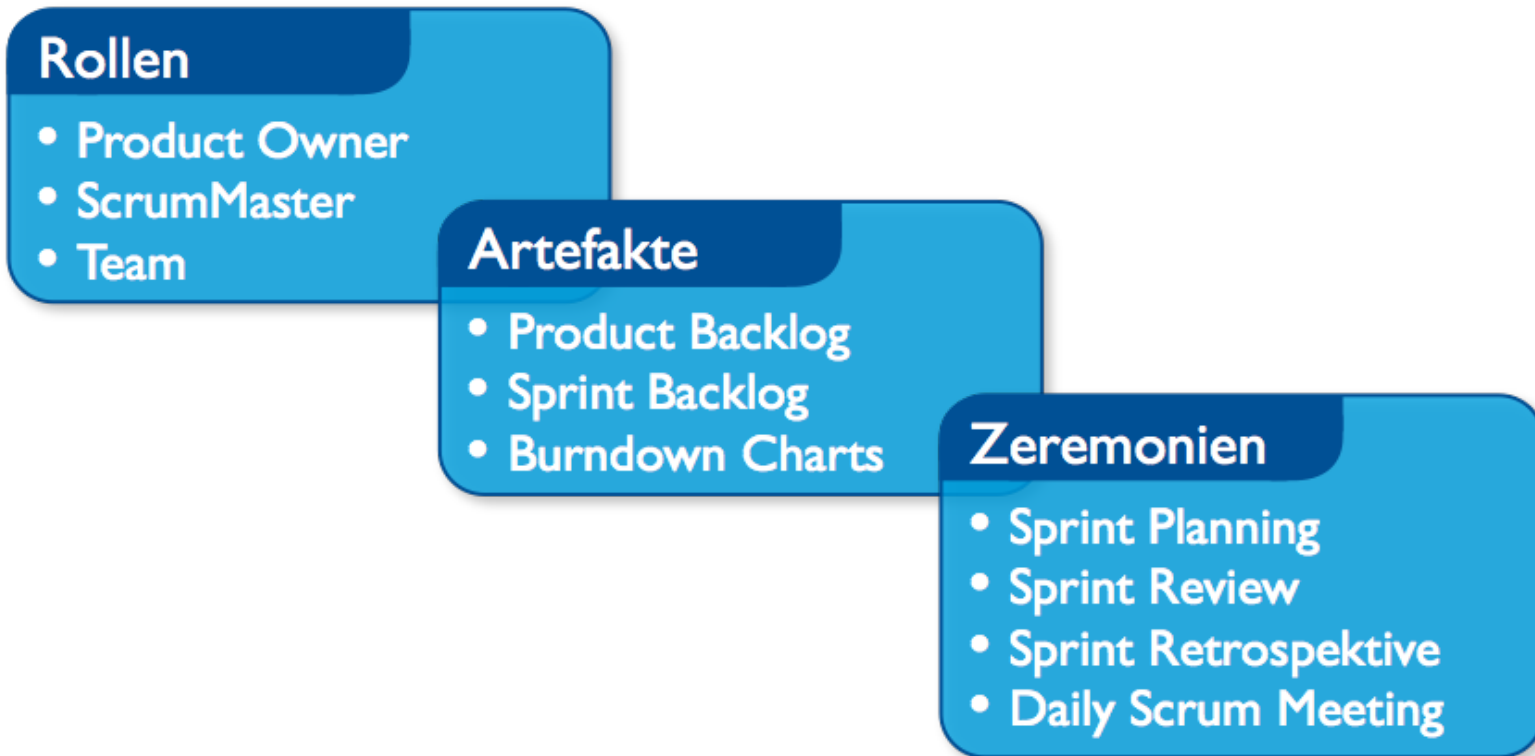
Scrum

Überblick



Scrum

Überblick



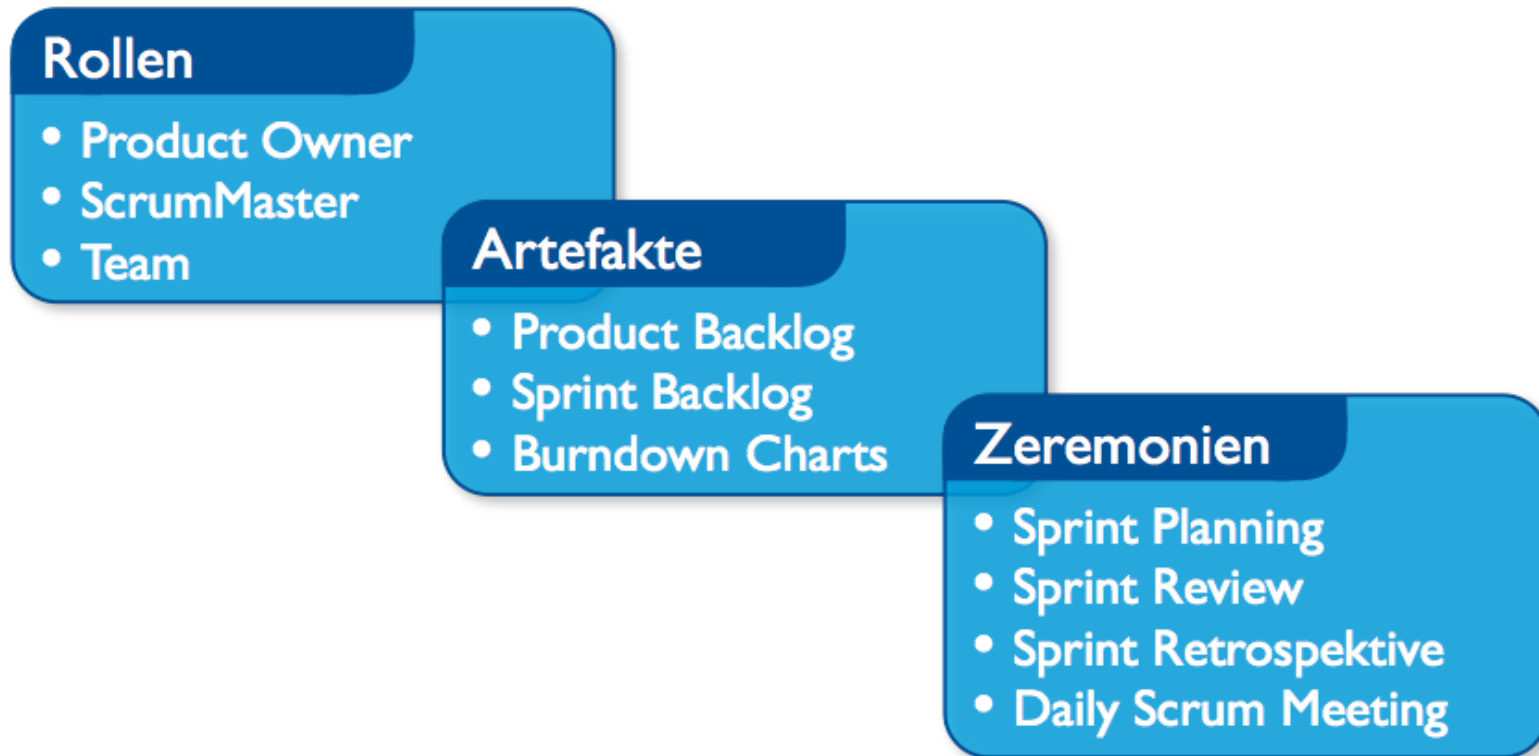
Scrum

Überblick



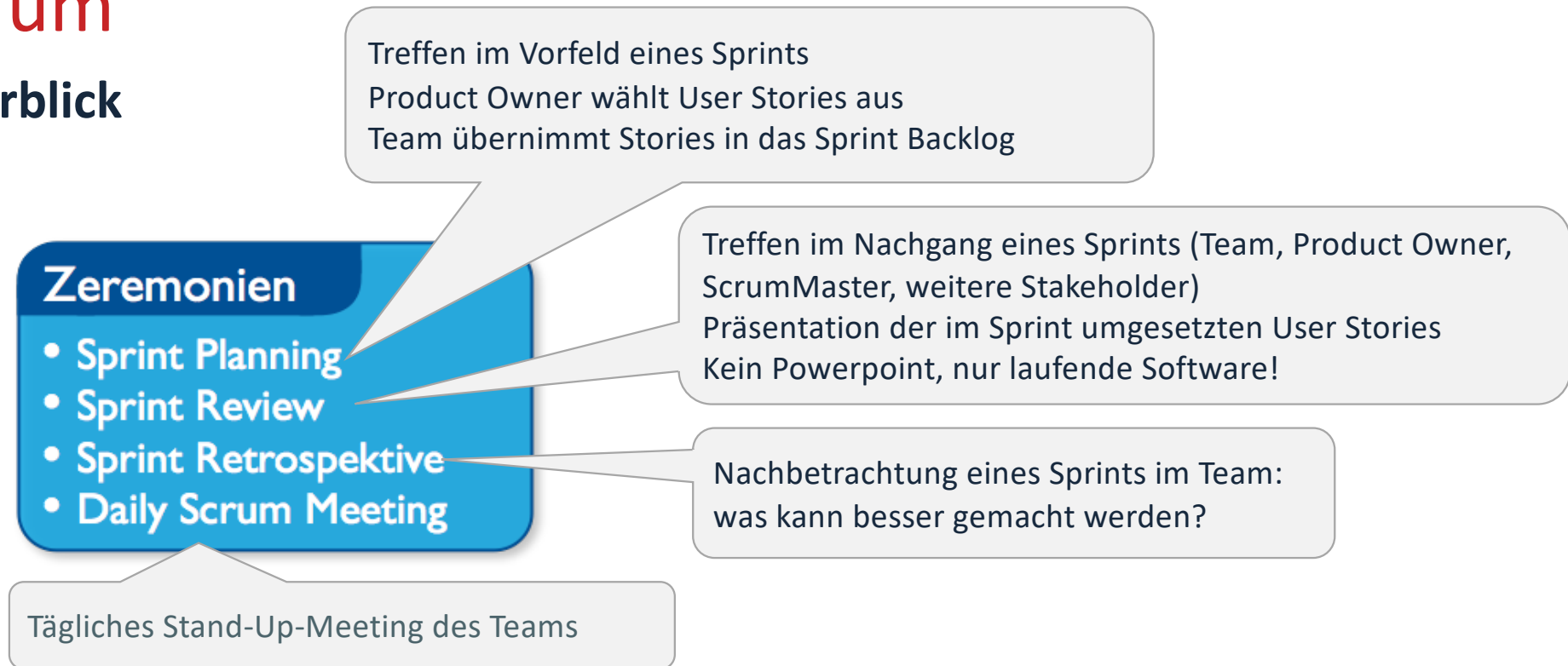
Scrum

Überblick



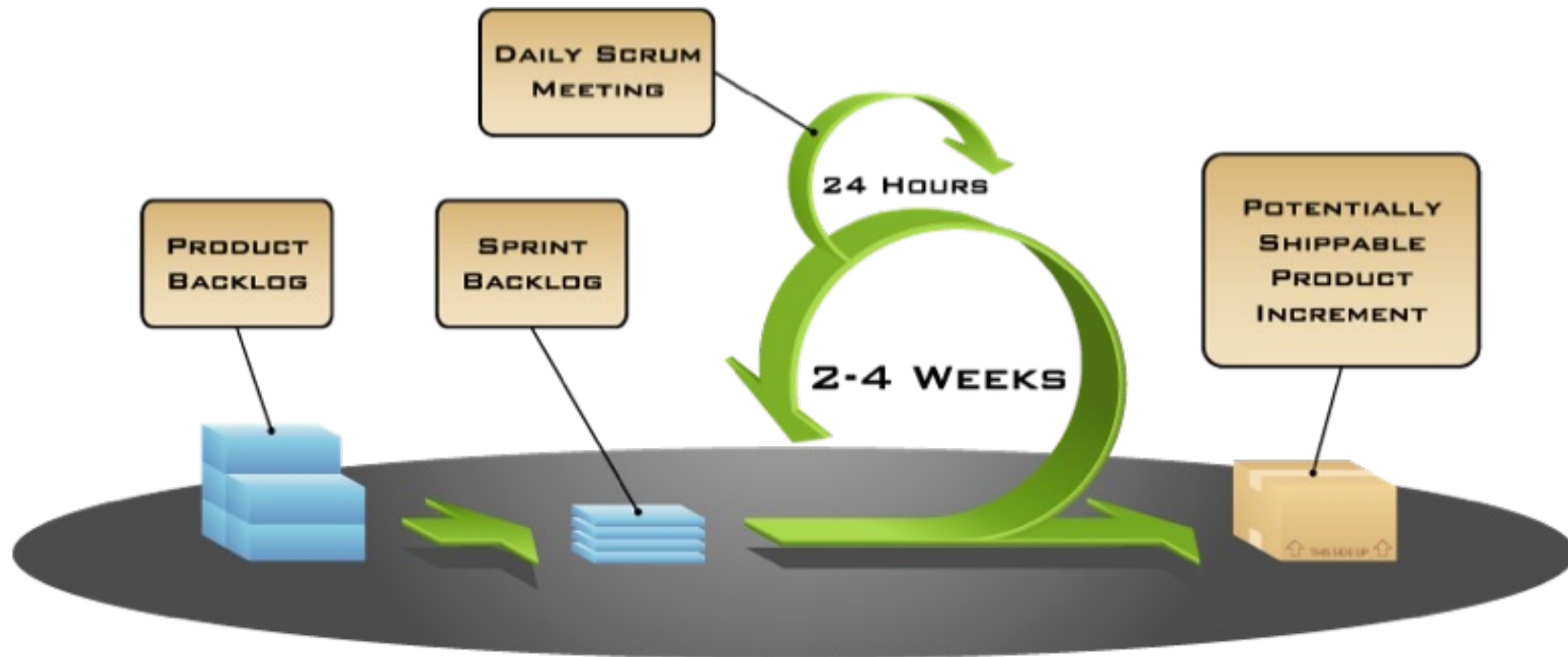
Scrum

Überblick



Scrum

Überblick



Scrum

Product Backlog

Im **Product Backlog** sind alle User Stories mit einer eindeutigen ID aufgelistet.

Der **Produkt Owner** priorisiert die User Stories. Stories mit hohem Wert werden nach oben gesetzt.

Id	Beschreibung	Story Points	Priorität
1	Als Modulverantwortlicher will ich Modulbeschreibungen suchen	5	200
2	Als Modulverantwortlicher will ich Modulbeschreibungen ändern	8	170
3	Als Student will ich das Modulhandbuch ansehen	5	160
4	Freigabeworkflow	40	90

Das Team liefert eine Aufwandsschätzung in Form von Story Points

Scrum

Product Backlog

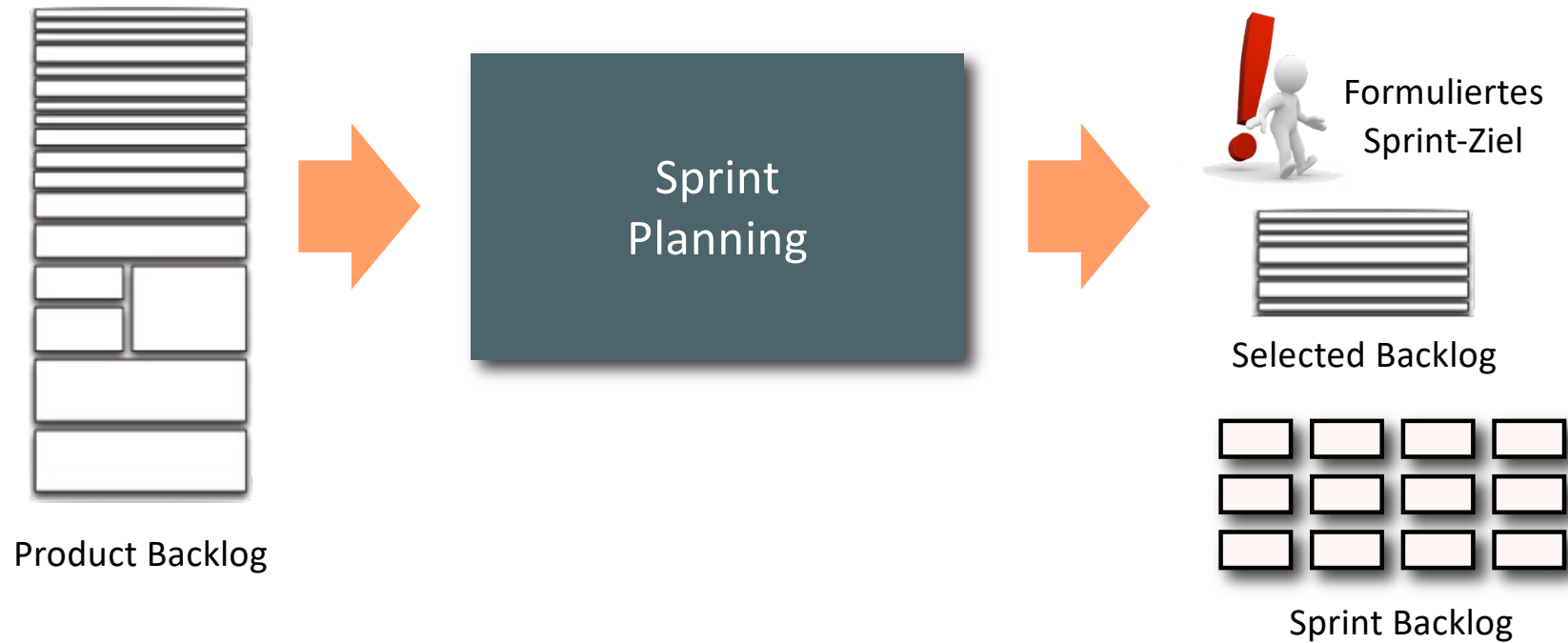
Stories mit hoher Priorität müssen klein genug sein, um im nächsten Sprint umgesetzt werden zu können. Stories mit geringer Priorität können noch unspezifisch und groß sein.

In Kürze umzusetzende User-Stories müssen ggf. aufgeteilt/geschnitten werden, um einen akzeptable Größe zu erhalten. Das ist Aufgabe des **Product Owner**.



Scrum

Sprint Planning

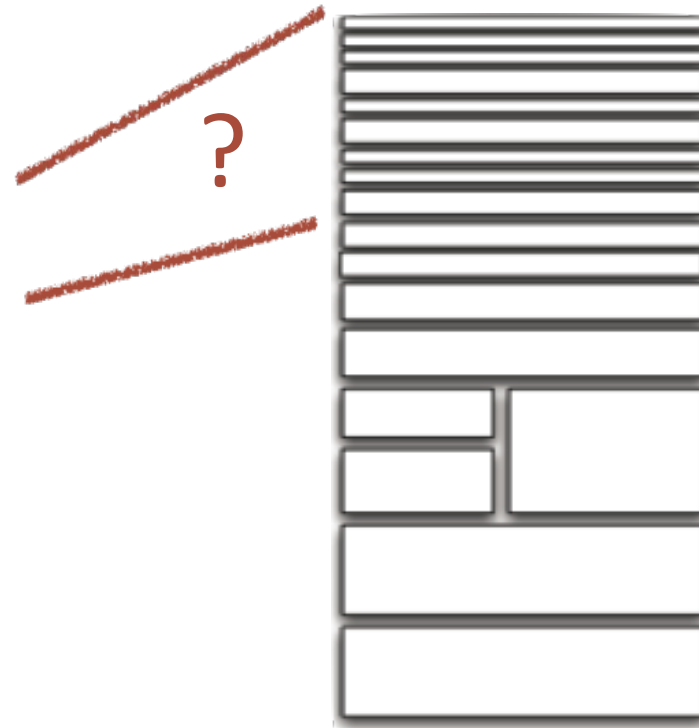


Scrum

Sprint Planning



Wie viele User
Stories passen
in den
nächsten
Sprint?



Scrum

Sprint Planning



- Vor dem Sprint Planning ermittelt der Scrum Master die **Velocity** des Teams
- Schätzung der im Sprint erreichbaren Storypoints
- Basis ist der Durchschnittswert aus vergangenen Sprints
- Voraussetzung:
 - ✓ Unverändertes Team
 - ✓ Gleiche Sprintlänge

Scrum

Daily Scrum Meeting

- Tägliches Stand-Up-Meeting des Teams während des Sprints
- Moderation durch den Scrum Master
- Drei Fragen werden beantwortet:
 - Was habe ich seit dem letzten Daily Scrum erreicht?
 - Was plane ich bis zum nächsten Daily Scrum?
 - Was behindert mich?
- Aktualisierung des Taskboards

Scrum

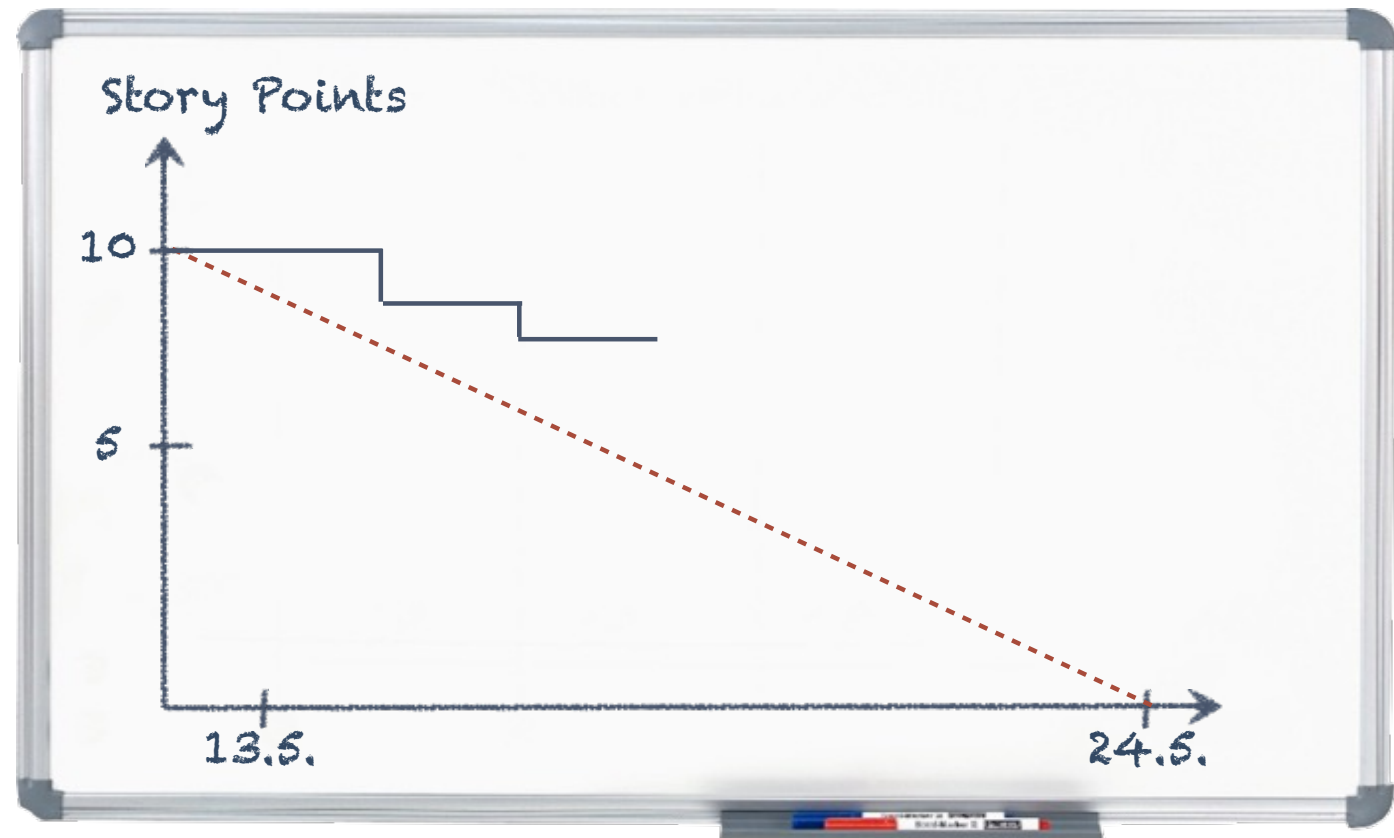
Taskboard

Story	To Do		In Process	To Verify	Done
As a user, I... 8 points	Code the... 9	Test the... 8	Code the... DC 4	Test the... SC 6	Code the... D
	Code the... 2	Code the... 8	Test the... SC 8		Test the... SC 8
	Test the... 8	Test the... 4			Test the... SC
As a user, I... 5 points	Code the... 8	Test the... 8	Code the... DC 8		Test the... SC
	Code the... 4	Code the... 6			Test the... SC 6

ohm

Scrum

Burn Down Chart



Scrum

Sprint Abschluss

Sprint Review

Wer?	Product Owner, Scrum Master, Team, interessierte Stakeholder
Wann?	Am Ende eines jeden Sprints Einladung bereits bei Sprintstart
Warum?	Um die abgeschlossenen Stories zu präsentieren und Feedback einzuholen

Scrum

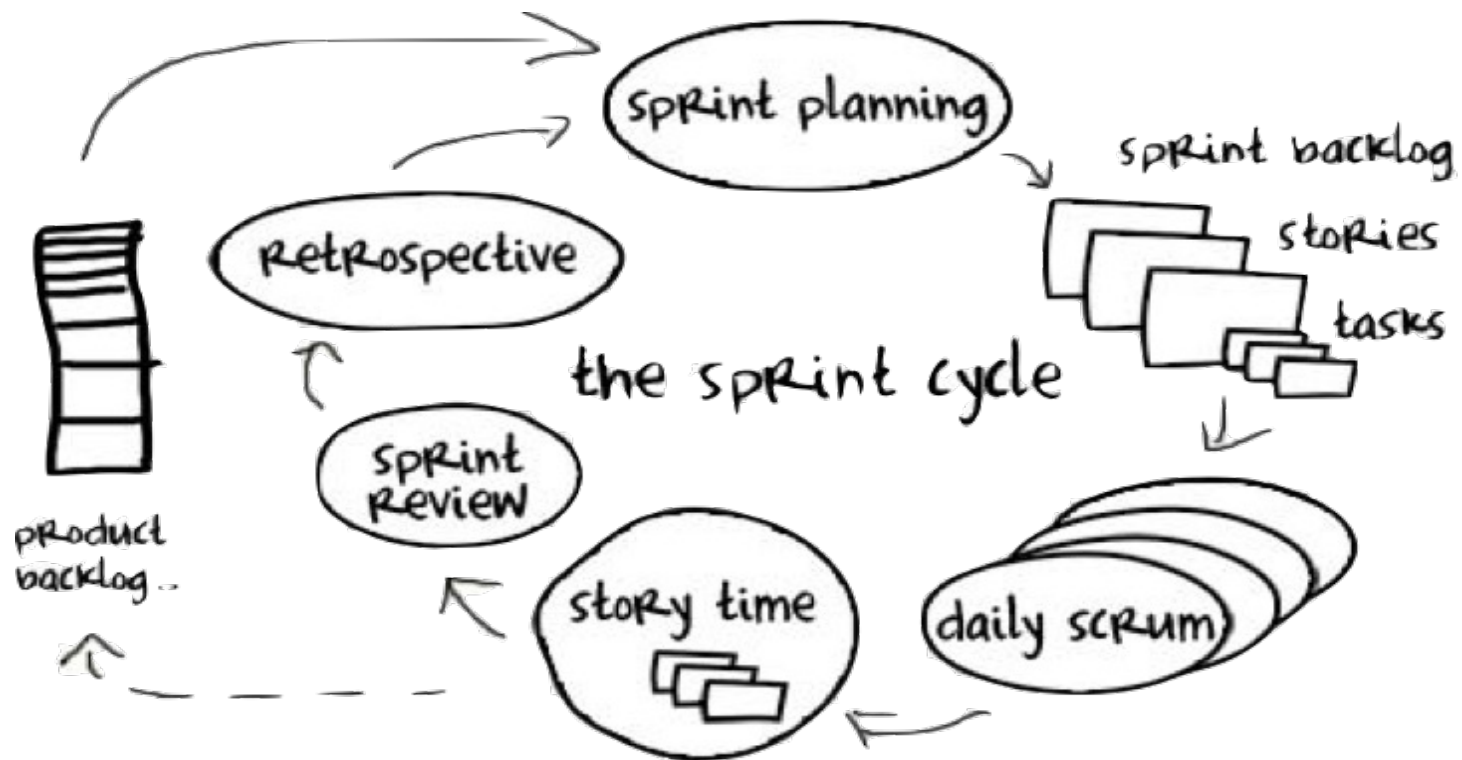
Sprint Abschluss

Sprint Retrospektive

Wer?	Product Owner, Scrum Master, Team
Wann?	Am Ende eines jeden Sprints Einladung bereits bei Sprintstart
Warum?	Um die Rahmenbedingungen und Prozesse zu verbessern und die Performance zu optimieren.

Scrum

Zusammenfassung



Vielen Dank für die Aufmerksamkeit!

Fragen?